User Input

Generated from .github/agents/speckit.clarify.agent.md

Artifact type: other

Table of contents

Body

User Input

```text $ARGUMENTS ``` You **MUST** consider the user input before proceeding (if not empty).

Pre-Execution Checks

Check for extension hooks (before clarification): - Check if `.specify/extensions.yml` exists in the project root. - If it exists, read it and look for entries under the `hooks.before_clarify` key - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally - Filter out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook `condition` expressions: - If the hook has no `condition` field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty `condition`, skip the hook and leave condition evaluation to the HookExecutor implementation - For each executable hook, output the following based on its `optional` flag: - **Optional hook** (`optional: true`): ``` ## Extension Hooks **Optional Pre-Hook**: {extension} Command: `/{command}` Description: {description} Prompt: {prompt} To execute: `/{command}` ``` - **Mandatory hook** (`optional: false`): ``` ## Extension Hooks **Automatic Pre-Hook**: {extension} Executing: `/{command}` EXECUTE_COMMAND: {command} Wait for the result of the hook command before proceeding to the Outline. ``` After emitting the block above you MUST actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal `{command}`
```

id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or `$speckit-...`). Emitting the block alone does not run the hook. - If no hooks are registered or `.specify/extensions.yml` does not exist, skip silently

Outline

Goal: Detect and reduce ambiguity or missing decision points in the active feature specification and record the clarifications directly in the spec file. Note: This clarification workflow is expected to run (and be completed) BEFORE invoking `/speckit.plan`. If the user explicitly states they are skipping clarification (e.g., exploratory spike), you may proceed, but must warn that downstream rework risk increases. Execution steps: 1. Run `.specify/scripts/powershell/check-prerequisites.ps1 -Json -PathsOnly` from repo root **once** (combined `--json --paths-only` mode / `-Json -PathsOnly`). Parse minimal JSON payload fields: - `FEATURE_DIR` - `FEATURE_SPEC` - (Optionally capture `IMPL_PLAN`, `TASKS` for future chained flows.) - If JSON parsing fails, abort and instruct user to re-run `/speckit.specify` or verify feature branch environment. - For single quotes in args like "I'm Groot", use escape syntax: e.g 'I'\''m Groot' (or double-quote if possible: "I'm Groot"). 2. **IF EXISTS**: Load `.specify/memory/constitution.md` for project principles and governance constraints. 3. Load the current spec file. Perform a structured ambiguity & coverage scan using this taxonomy. For each category, mark status: Clear / Partial / Missing. Produce an internal coverage map used for prioritization (do not output raw map unless no questions will be asked). Functional Scope & Behavior: - Core user goals & success criteria - Explicit out-of-scope declarations - User roles / personas differentiation Domain & Data Model: - Entities, attributes, relationships - Identity & uniqueness rules - Lifecycle/state transitions - Data volume / scale assumptions Interaction & UX Flow: - Critical user journeys / sequences - Error/empty/loading states - Accessibility or localization notes Non-Functional Quality Attributes: - Performance (latency, throughput targets) - Scalability (horizontal/vertical, limits) - Reliability & availability (uptime, recovery expectations) - Observability (logging, metrics, tracing signals) - Security & privacy (authN/Z, data protection, threat assumptions) - Compliance / regulatory constraints (if any) Integration & External Dependencies: - External services/APIs and failure modes - Data import/export formats - Protocol/versioning assumptions Edge Cases & Failure Handling: - Negative scenarios - Rate limiting / throttling - Conflict resolution (e.g., concurrent edits) Constraints & Tradeoffs: - Technical constraints (language, storage, hosting) - Explicit tradeoffs or rejected alternatives Terminology & Consistency: - Canonical glossary terms - Avoided synonyms / deprecated terms Completion Signals: - Acceptance criteria testability - Measurable Definition of Done style indicators Misc / Placeholders: - TODO markers / unresolved decisions - Ambiguous adjectives ("robust", "intuitive") lacking quantification For each category with Partial or Missing status, add a candidate question opportunity unless: - Clarification would not materially change implementation or validation strategy - Information is better deferred to planning phase (note internally) 4. Generate (internally) a prioritized queue of candidate clarification questions (maximum 5). Do NOT output them all at once. Apply these constraints: - Maximum of 5 total questions across the whole session. - Each question must be answerable with EITHER: - A short multiple■choice selection (2–5 distinct, mutually exclusive options), OR - A one-word / short■phrase answer (explicitly constrain: "Answer in <=5 words"). - Only include questions

whose answers materially impact architecture, data modeling, task decomposition, test design, UX behavior, operational readiness, or compliance validation. - Ensure category coverage balance: attempt to cover the highest impact unresolved categories first; avoid asking two low-impact questions when a single high-impact area (e.g., security posture) is unresolved. - Exclude questions already answered, trivial stylistic preferences, or plan-level execution details (unless blocking correctness). - Favor clarifications that reduce downstream rework risk or prevent misaligned acceptance tests. - If more than 5 categories remain unresolved, select the top 5 by (Impact * Uncertainty) heuristic. 5. Sequential questioning loop (interactive): - Present EXACTLY ONE question at a time. - For multiple■choice questions: - **Analyze all options** and determine the **most suitable option** based on: - Best practices for the project type - Common patterns in similar implementations - Risk reduction (security, performance, maintainability) - Alignment with any explicit project goals or constraints visible in the spec - Present your **recommended option prominently** at the top with clear reasoning (1-2 sentences explaining why this is the best choice). - Format as: `**Recommended:** Option [X] - ` - Then render all options as a Markdown table: | Option | Description | |--------|-------------| | A | | | B | | | C | (add D/E as needed up to 5) | | Short | Provide a different short answer (<=5 words) (Include only if free-form alternative is appropriate) | - After the table, add: `You can reply with the option letter (e.g., "A"), accept the recommendation by saying "yes" or "recommended", or provide your own short answer.` - For short■answer style (no meaningful discrete options): - Provide your **suggested answer** based on best practices and context. - Format as: `**Suggested:** - ` - Then output: `Format: Short answer (<=5 words). You can accept the suggestion by saying "yes" or "suggested", or provide your own answer.` - After the user answers: - If the user replies with "yes", "recommended", or "suggested", use your previously stated recommendation/suggestion as the answer. - Otherwise, validate the answer maps to one option or fits the <=5 word constraint. - If ambiguous, ask for a quick disambiguation (count still belongs to same question; do not advance). - Once satisfactory, record it in working memory (do not yet write to disk) and move to the next queued question. - Stop asking further questions when: - All critical ambiguities resolved early (remaining queued items become unnecessary), OR - User signals completion ("done", "good", "no more"), OR - You reach 5 asked questions. - Never reveal future queued questions in advance. - If no valid questions exist at start, immediately report no critical ambiguities. 6. Integration after EACH accepted answer (incremental update approach): - Maintain in-memory representation of the spec (loaded once at start) plus the raw file contents. - For the first integrated answer in this session: - Ensure a `## Clarifications` section exists (create it just after the highest-level contextual/overview section per the spec template if missing). - Under it, create (if not present) a `### Session YYYY-MM-DD` subheading for today. - Append a bullet line immediately after acceptance: `- Q: → A: `. - Then immediately apply the clarification to the most appropriate section(s): - Functional ambiguity → Update or add a bullet in Functional Requirements. - User interaction / actor distinction → Update User Stories or Actors subsection (if present) with clarified role, constraint, or scenario. - Data shape / entities → Update Data Model (add fields, types, relationships) preserving ordering; note added constraints succinctly. - Non-functional constraint → Add/modify measurable criteria in Success Criteria > Measurable Outcomes (convert vague adjective to metric or explicit target). - Edge case / negative flow → Add a new bullet under Edge Cases / Error Handling (or create such subsection if template provides placeholder for

it). - Terminology conflict → Normalize term across spec; retain original only if necessary by adding `(formerly referred to as "X")` once. - If the clarification invalidates an earlier ambiguous statement, replace that statement instead of duplicating; leave no obsolete contradictory text. - Save the spec file AFTER each integration to minimize risk of context loss (atomic overwrite). - Preserve formatting: do not reorder unrelated sections; keep heading hierarchy intact. - Keep each inserted clarification minimal and testable (avoid narrative drift). 7. Validation (performed after EACH write plus final pass): - Clarifications session contains exactly one bullet per accepted answer (no duplicates). - Total asked (accepted) questions ≤ 5. - Updated sections contain no lingering vague placeholders the new answer was meant to resolve. - No contradictory earlier statement remains (scan for now-invalid alternative choices removed). - Markdown structure valid; only allowed new headings: `## Clarifications`, `### Session YYYY-MM-DD`. - Terminology consistency: same canonical term used across all updated sections. 8. Write the updated spec back to `FEATURE_SPEC`. 9. **Re-validate Spec Quality Checklist** (if it exists): - Check if `FEATURE_DIR/checklists/requirements.md` exists. - If it does NOT exist, skip this step silently. - If it exists: 1. Read the checklist file. 2. Identify all GitHub task-list checkbox lines — lines matching `- [ ]`, `- [x]`, or `- [X]` (case-insensitive, tolerant of leading whitespace for nested items) outside of code fences. Ignore all other content (headings, notes, non-checkbox bullets, metadata). 3. For each checkbox line, record its current marker state (checked or unchecked) and item text into a before-snapshot list. 4. Re-evaluate each checkbox item against the **updated** spec (the version just saved in step 7). 5. For each checkbox item, update only if the checked/unchecked state actually changes: - If the item now passes and was unchecked: change `[ ]` to `[x]`. - If the item now fails and was checked: change `[x]`/`[X]` to `[ ]`. - If the state is unchanged: leave the marker as-is (preserve existing case to avoid cosmetic diffs). 6. Save the updated checklist file. **Only toggle the `[ ]`/`[x]` marker portion of checkbox lines whose state changed.** All other file content — headings, metadata, notes, line ordering, whitespace — must remain unchanged to avoid noisy diffs. 7. Compare the before-snapshot with the current state to compute three lists for the Completion Report: - **Newly passing**: items that changed from unchecked to checked. - **Regressions**: items that changed from checked to unchecked. - **Still unchecked**: items that remain unchecked. 8. Record the before/after pass counts as checked/total checkbox items (e.g., "12/16 → 15/16 items passing"). Behavior rules: - If no meaningful ambiguities found (or all potential questions would be low-impact), respond: "No critical ambiguities detected worth formal clarification." and suggest proceeding. - If spec file missing, instruct user to run `/speckit.specify` first (do not create a new spec here). - Never exceed 5 total asked questions (clarification retries for a single question do not count as new questions). - Avoid speculative tech stack questions unless the absence blocks functional clarity. - Respect user early termination signals ("stop", "done", "proceed"). - If no questions asked due to full coverage, output a compact coverage summary (all categories Clear) then suggest advancing. - If quota reached with unresolved high-impact categories remaining, explicitly flag them under Deferred with rationale. Context for prioritization: $ARGUMENTS

Mandatory Post-Execution Hooks

You MUST complete this section before reporting completion to the user. Check if `.specify/extensions.yml` exists in the project root. - If it does not exist, or no hooks are registered under `hooks.after_clarify`, skip to the Completion Report. - If it exists, read it and look for entries under the `hooks.after_clarify` key. - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue to the Completion Report. - Filter out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook `condition` expressions: - If the hook has no `condition` field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty `condition`, skip the hook and leave condition evaluation to the HookExecutor implementation - For each executable hook, output the following based on its `optional` flag: - **Mandatory hook** (`optional: false`) — **You MUST emit `EXECUTE_COMMAND:` for each mandatory hook**: ``` ## Extension Hooks **Automatic Hook**: {extension} Executing: `/{command}` EXECUTE_COMMAND: {command} ``` After emitting the block above you MUST actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal `{command}` id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or `$speckit-...`). Emitting the block alone does not run the hook. - **Optional hook** (`optional: true`): ``` ## Extension Hooks **Optional Hook**: {extension} Command: `/{command}` Description: {description} Prompt: {prompt} To execute: `/{command}` ```

Completion Report

Report completion (after questioning loop ends or early termination): - Number of questions asked & answered. - Path to updated spec. - Sections touched (list names). - Spec quality checklist status (if `FEATURE_DIR/checklists/requirements.md` was re-validated): show before/after pass counts (e.g., "Spec Quality Checklist: 12/16 → 15/16 items passing") and list any items that changed state — both newly checked (unchecked → checked) and any regressions (checked → unchecked). If any items remain unchecked, list them as areas needing attention. - Coverage summary table listing each taxonomy category with Status: Resolved (was Partial/Missing and addressed), Deferred (exceeds question quota or better suited for planning), Clear (already sufficient), Outstanding (still Partial/Missing but low impact). - If any Outstanding or Deferred remain, recommend whether to proceed to `/speckit.plan` or run `/speckit.clarify` again later post-plan. - Suggested next command.

Done When

- [ ] Spec ambiguities identified and clarifications integrated into spec file - [ ] Spec quality checklist re-validated against updated spec (if `FEATURE_DIR/checklists/requirements.md` exists) - [ ] Extension hooks dispatched or skipped according to the rules in Mandatory Post-Execution Hooks above - [ ] Completion reported to user with questions answered, sections touched, checklist status, and coverage summary